Software Requirements Specification (SRS)

Citizen Participation-Based Coastal Disaster Reporting & Crisis Management System

Submitted By:

Name: Sumon Chandra Barman **ID:** 220212

Name: Aditi Debnath **ID:** 220224

Bangladesh Coastal Disaster Management Context

Contents

1 Introduction

Bangladesh is highly exposed to coastal hazards because of its low-lying deltaic geography, dense coastal population, river systems, cyclones, storm surges, tidal flooding, salinity intrusion, erosion and climate-related extreme events. Coastal communities often experience the earliest impacts of disasters, while government agencies, local authorities, NGOs, volunteers and emergency responders need timely and location-specific information to make decisions.

The Citizen Participation-Based Coastal Disaster Reporting & Crisis Management System is proposed as a digital platform through which citizens can report hazards and disaster impacts, receive verified warnings and instructions, request or identify emergency assistance, and communicate relevant situational information. The system will combine citizen-generated reports with administrative verification, geographic information, official alerts and crisis-management workflows.

The system is intended to support—not replace—existing government disaster-management structures. It should complement national and local mechanisms involving agencies such as the Bangladesh Meteorological Department (BMD), Department of Disaster Management (DDM), local administration, city/municipal authorities, Union Parishads, NGOs, volunteers and community organizations.

2 Project Overview

2.1 Proposed System

The system will provide web and mobile-friendly interfaces for citizens, volunteers/responders, local administrators, disaster-management authorities and system administrators. Citizens will be able to submit structured disaster reports with location, category, description, severity, timestamp and optional photos/videos. Authorized personnel will verify, prioritize and manage reports through an operational dashboard.

2.2 Main Objectives

- Enable rapid, structured and location-aware citizen disaster reporting.
- Improve situational awareness for coastal disaster response.
- Provide verified disaster alerts, warnings and preparedness instructions.
- Prioritize incidents according to severity, urgency, affected population and location.
- Support communication between citizens, responders, volunteers and authorities.
- Provide maps and dashboards for incident visualization and resource coordination.
- Maintain an auditable history of reports, verification, decisions and response actions.
- Generate analytical reports to support post-disaster assessment and future preparedness.

2.3 Target Users

User Role	Main Responsibilities
Citizen	Report incidents, request help, view verified alerts, track submitted reports.
Community Volunteer	Assist with local verification, response coordination and field updates.
Responder/Field Officer	Review assigned incidents, update response status and record field actions.

User Role	Main Responsibilities
Local Authority	Monitor incidents in assigned geographic areas and coordinate response.
Disaster Management Officer	Verify/prioritize incidents, issue operational instructions and monitor crisis status.
System Administrator	Manage users, roles, categories, configurations, security and audit logs.

2.4 High-Level Workflow

Citizen/Field Input → Report Submission → Location & Data Validation → Verification/Triage → Severity & Priority Assessment → Assignment → Response/Coordination → Status Updates → Resolution → Post-Incident Analysis

3 Purpose of the Document

This Software Requirements Specification defines the functional, business, non-functional and technical requirements of the proposed Citizen Participation-Based Coastal Disaster Reporting & Crisis Management System. It provides a common reference for stakeholders, developers, designers, testers, project supervisors and future maintainers.

The document is written as an editable baseline. Project-specific values such as exact authority names, SLA targets, hosting provider, database choice, supported languages and notification channels may be adjusted during implementation.

4 Glossary of Terms

Term	Definition
Citizen Report	A disaster/hazard report submitted by a citizen through the system.
Incident	A reported event or situation requiring monitoring, verification or response.
Crisis	A serious situation requiring coordinated decisions and emergency response.
Hazard	A potentially damaging event such as cyclone, flood, storm surge or landslide.
Coastal Disaster	A disaster affecting coastal areas, including cyclones, storm surges, tidal flooding and erosion.
Verification	The process of checking whether a report is credible and sufficiently supported.
Triage	Initial assessment used to categorize and prioritize incoming incidents.
Severity	Estimated seriousness or impact level of an incident.
Priority	Operational urgency assigned to an incident.
Geolocation	Latitude and longitude or an equivalent geographic representation of a report.
Geofencing	Defining geographic boundaries for alerts, monitoring or access control.
SOS/Help Request	A citizen request indicating a need for urgent assistance.

Term	Definition
Responder	Authorized person or organization responsible for taking response actions.
Dashboard	A visual interface showing operational information, statistics and maps.
GIS	Geographic Information System used to visualize and analyze location-based data.
RBAC	Role-Based Access Control for managing permissions according to user roles.
Audit Log	Chronological record of important system and user actions.
SLA	Service Level Agreement defining expected response or service targets.
API	Application Programming Interface used for communication between software components.

5 Business Requirements

5.1 Business Goals

ID	Requirement
BR-01	The system shall improve the speed and quality of coastal disaster information collection.
BR-02	The system shall enable citizens to participate directly in disaster reporting and situational awareness.
BR-03	The system shall support authorities in identifying high-priority incidents.
BR-04	The system shall reduce information fragmentation by maintaining incidents in a centralized platform.
BR-05	The system shall support location-based crisis monitoring and response coordination.
BR-06	The system shall provide transparent status tracking for submitted reports where appropriate.
BR-07	The system shall support evidence-based post-disaster assessment and planning.
BR-08	The system shall be usable in coastal communities with varying digital literacy and network quality.

5.2 Bangladesh-Specific Business Considerations

- The design should support cyclone-prone coastal districts and island/char areas where communication may be intermittent.
- Location-based communication should consider unions, upazilas, districts and other locally meaningful geographic units.
- The platform should complement, rather than duplicate, official warning and emergency-response channels.
- Citizen reports should be treated as potentially unverified information until an appropriate verification process is completed.

- Privacy and access controls are essential because reports may contain personal information, phone numbers, photographs, medical/emergency details or sensitive locations.
- The system should support low-bandwidth operation and graceful handling of temporary network loss.

6 Functional Requirements

6.1 User Registration & Authentication

- FR-01: The system shall allow citizens to create an account using an approved authentication method.
- FR-02: The system shall support secure login, logout and password/account recovery.
- FR-03: The system shall support role-based access for citizen, volunteer, responder, authority and administrator users.
- FR-04: The system shall allow administrators to activate, suspend or deactivate accounts according to policy.

6.2 Citizen Disaster Reporting

- FR-06: Citizens shall be able to submit a disaster/incident report.
- FR-07: A report shall support incident category, description, severity, date/time and location.
- FR-08: The system should allow photo/video or other evidence attachments subject to configured limits.
- FR-09: The system shall capture report status such as Submitted, Under Review, Verified, Assigned, In Progress, Resolved or Rejected.
- FR-10: The system shall prevent or flag clearly invalid/incomplete submissions.
- FR-11: Citizens shall be able to view the status/history of their own reports.
- FR-12: The system should support duplicate-report detection or flagging for similar nearby incidents.

6.3 Disaster & Hazard Categories

- FR-13: Administrators shall be able to configure disaster categories.
- FR-14: Initial categories may include Cyclone, Storm Surge, Tidal Flooding, River Flooding, Flash Flooding, Coastal Erosion, Waterlogging, Landslide, Embankment Breach, Lightning, Fire, Infrastructure Damage, Missing Person and Other Emergency.

6.4 Location & Mapping

- FR-15: The system shall support GPS-based location capture where available.
- FR-16: Users shall be able to select a location manually when GPS is unavailable or inaccurate.
- FR-17: Reports shall be visualized on a map for authorized operational users.
- FR-18: The system should associate reports with administrative geographic units where possible.
- FR-19: Authorized users shall be able to filter incidents by location, category, severity, priority and status.

6.5 Verification & Triage

- FR-20: Authorized personnel shall be able to review incoming reports.
- FR-21: Reviewers shall be able to mark reports as Verified, Unverified, Rejected or Duplicate

according to workflow rules.

- FR-22: The system shall support priority levels such as Critical, High, Medium and Low.
- FR-23: Priority may consider severity, number of affected people, proximity to critical infrastructure, geographic risk and time sensitivity.
- FR-24: Verification actions shall be recorded in the audit trail.

6.6 Crisis Management & Response

- FR-25: Authorized officers shall be able to assign incidents to responders or response teams.
- FR-26: Responders shall be able to update incident progress and field status.
- FR-27: The system shall support resource/assistance records such as shelter information, rescue requests, medical assistance and food/water requirements.
- FR-28: Authorized users shall be able to record response actions and outcomes.
- FR-29: The system shall support escalation of unresolved or critical incidents.

6.7 Alerts & Notifications

- FR-30: Authorized authorities shall be able to publish verified alerts and preparedness instructions.
- FR-31: The system shall support location-targeted notifications where technically and legally appropriate.
- FR-32: Notifications may use in-app notifications, SMS, email or push notifications depending on available infrastructure.
- FR-33: Critical alerts shall clearly identify the alert type, affected area, time and recommended action.
- FR-34: The system shall distinguish official/verified alerts from citizen-generated unverified reports.

6.8 Dashboard & Analytics

- FR-35: The system shall provide dashboards for authorized users.
- FR-36: Dashboards shall show active incidents, severity distribution, geographic distribution and response status.
- FR-37: Authorized users shall be able to filter and export operational reports.
- FR-38: The system should provide historical statistics for disaster preparedness and post-event analysis.

6.9 Administration & Audit

- FR-39: Administrators shall manage users, roles, permissions and configurable system data.
- FR-40: The system shall maintain audit logs for security-sensitive and operationally significant actions.
- FR-41: Administrators shall be able to manage notification templates and report categories.
- FR-42: The system shall support backup and recovery procedures.

7 Non-functional Requirements

ID	Category	Requirement
NFR-01	Performance	Normal user actions should return within an agreed target under expected load.
NFR-02	Availability	The system should remain available during disaster periods with appropriate redundancy and monitoring.
NFR-03	Scalability	The architecture should support growth in users, reports, attachments and concurrent incidents.
NFR-04	Security	Authentication, authorization, encryption and secure session/token management shall be implemented.
NFR-05	Privacy	Personal data shall be collected minimally and exposed only to authorized users.
NFR-06	Reliability	The system shall protect report integrity and prevent accidental loss of submitted data.
NFR-07	Usability	Citizen-facing workflows should be simple, mobile-friendly and understandable to users with limited technical literacy.
NFR-08	Accessibility	Interfaces should follow practical accessibility principles, including readable text, clear labels and sufficient interaction targets.
NFR-09	Low Bandwidth	The application should minimize payload size and support intermittent connectivity where feasible.
NFR-10	Maintainability	The codebase shall be modular, documented and testable.
NFR-11	Auditability	Important actions and status changes shall be traceable through audit logs.
NFR-12	Disaster Recovery	The system shall have defined backup, restoration and recovery procedures.
NFR-13	Data Integrity	Validation, transaction handling and database constraints shall reduce inconsistent or duplicate data.
NFR-14	Interoperability	The system should expose documented APIs for approved integrations.
NFR-15	Monitoring	Operational health, errors, performance and critical security events should be monitored.

8 Technical Requirements

8.1 Proposed Architecture

A modular client-server architecture is recommended. A mobile/web client communicates with a backend REST API. The backend handles authentication, business rules, reporting workflows, notifications and integrations. A relational database stores structured data, while object/file storage manages photos and other attachments. A map/GIS service provides geographic visualization.

8.2 Suggested Technology Stack

Layer	Suggested Technology	Notes
Frontend	React.js / Next.js or responsive PWA	Citizen and administrative interfaces.

Layer	Suggested Technology	Notes
Mobile	PWA initially; Flutter/React Native if native features are required	Useful for GPS, camera and offline workflows.
Backend	Django + Django REST Framework	Suitable for RBAC, APIs, admin workflows and rapid development.
Database	PostgreSQL + PostGIS	Recommended for relational and geographic data.
Cache/Queue	Redis	Caching, rate limiting and background jobs.
Background Jobs	Celery	Notifications, media processing and scheduled tasks.
Storage	S3-compatible object storage	Photos/videos/documents; configurable by deployment.
Maps	OpenStreetMap-based solution / approved GIS provider	Final provider depends on budget, licensing and availability.
Deployment	Linux server / cloud platform	Should support scaling and monitoring.
Monitoring	Application logs + metrics + error tracking	Required for operational reliability.
API Format	REST/JSON over HTTPS	Document using OpenAPI/Swagger.

8.3 Core Data Entities

- User
- Role
- Citizen Profile
- Volunteer/Responder Profile
- Incident Report
- Incident Category
- Incident Location
- Evidence Attachment
- Verification Record
- Response Assignment
- Response Action
- Alert
- Notification
- Shelter/Resource
- Administrative Area
- Audit Log

8.4 Security Requirements

- All production API traffic shall use HTTPS.
- Passwords shall never be stored in plaintext; a strong password-hashing algorithm shall be used.
- Role-based authorization shall be enforced server-side, not only in the frontend.
- Sensitive API endpoints shall implement authentication, authorization, validation and rate

limiting.
- File uploads shall be validated by type, size and security controls.
- Personally identifiable information shall not be unnecessarily exposed through public maps or APIs.
- Audit logs shall record relevant authentication, permission, verification and administrative events.
- Secrets, API keys and database credentials shall be stored outside source code and managed through environment/secret configuration.

8.5 API Requirements

- Authentication and user-management endpoints.
- Incident creation, retrieval, update and status endpoints.
- Location and geographic filtering endpoints.
- Verification and assignment endpoints for authorized users.
- Alert and notification endpoints.
- Dashboard/analytics endpoints.
- Attachment upload endpoints with secure validation.
- API documentation through OpenAPI/Swagger.
- Pagination, filtering, sorting and rate limiting for appropriate endpoints.

8.6 Testing Requirements

- Unit testing for core business logic.
- API/integration testing for major workflows.
- Role and permission testing.
- Validation and file-upload security testing.
- Performance/load testing for expected peak conditions.
- Usability testing with representative citizen and responder workflows.
- Offline/intermittent-network testing where supported.
- Security testing including authentication, authorization and common web vulnerabilities.

8.7 Future Enhancements

- AI-assisted duplicate/false-report detection with human verification.
- Automated incident prioritization using historical and real-time data.
- Integration with official weather/disaster alert APIs where authorized.
- Satellite/drone imagery integration for post-disaster assessment.
- Predictive risk maps for cyclone, flood and storm-surge exposure.
- USSD/SMS-based reporting for users without smartphones or reliable mobile data.
- Voice-based Bangla reporting for users with limited literacy.
- Community reputation/trust mechanisms for repeated reliable contributors.

A Suggested Incident Status Flow

Submitted → Under Review → Verified/Unverified → Prioritized → Assigned → In Progress → Resolved → Closed. Rejected and Duplicate may be terminal branches from the review stage.

Acknowledgement

This project was developed with the help of several tools and technologies, which supported different stages of design, development and documentation.

- **GitHub** – Used for version control, source code management and collaboration between team members throughout the development process.
- **AI Tools (e.g., Claude/ChatGPT)** – Used for assistance in requirement analysis, documentation drafting, debugging and generating structured technical content.
- **Django & React** – Used as the core backend and frontend frameworks for building and prototyping the proposed system.
- **Overleaf** – Used for writing, formatting and compiling this LaTeX-based documentation.
- **Draw.io / Figma** – Used for designing system diagrams, workflow charts and interface mockups.

We sincerely acknowledge the contribution of these tools, which made the development and documentation of this project more efficient and organized.